ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

The Lambda Calculus

> *"The λ calculus is, at heart, a simple notation for functions and applications. The main ideas are applying a function to an argument and forming functions by abstraction."*
> — Henk Barendregt

Why Study the λ Calculus?

Over the past three lectures, you've written code in Haskell, Scala, and Elixir. Despite their surface differences, you have noticed some striking similarities:

- **Anonymous functions**: `\x -> x + 1` (Haskell), `x => x + 1` (Scala), `fn x -> x + 1 end` (Elixir)
- **Higher-order functions**: `map`, `filter`, ... taking functions as arguments
- **Currying**: multi-argument functions as chains of single-argument functions
- **Immutability**: values that don't change, computation as transformation

These aren't coincidences. All three languages descend from a common ancestor: Alonzo Church's **λ calculus**, developed in the 1930s — before computers even existed.

The λ calculus is the *theoretical foundation* of functional programming.

- Every functional language is, at its core, the λ calculus plus conveniences (numbers, strings, pattern matching, types).
- Understanding the λ calculus means understanding the *essence* of functional programming, stripped to its mathematical core.

Church's work was motivated by the desire to capture the **computational** aspects of functions.

Unlike the set-theoretic view, where functions are characterized as sets of (*argument*, *value*) pairs, Church's intent was to create a **calculus** —a syntax for terms and a set of rewrite rules for transforming them—to capture the intuition about the behavior of functions, that is about not just *what* a function maps to *what*, but *how* the mapping is carried out.

The Untyped λ Calculus

We begin with the **untyped λ calculus** — a name chosen to distinguish it from typed variants we'll encounter later. Its minimalism is breathtaking: there are only three kinds of expressions.

Syntax

The syntax of λ expressions (also called λ *terms*) is defined as follows:

$$\begin{array}{rcll} M, N, P & ::= & x, y, z & \text{(variable)} \\ & | & \lambda x.M & \text{(abstraction)} \\ & | & M\ N & \text{(application)} \end{array}$$

That's it. Three constructs, and yet this tiny language is Turing-complete — capable of expressing any computable function.

Let's connect these to the languages we looked at in our lectures:

λ Calculus	Haskell	Scala	Elixir
x	`x`	`x`	`x`
$\lambda x.M$	`\x -> M`	`x => M`	`fn x -> M end`
$M\ N$	`M N`	`M(N)`	`M.(N)`

A **λ abstraction** $\lambda x.M$ is an anonymous function with formal parameter x and body M. The abstraction **binds** the variable x; we say x is in **scope** within M.

Notational Conventions

Before diving deeper, let's establish conventions that simplify our notation:

1. **Application is left-associative**: $M\ N\ P$ means $(M\ N)\ P$, not $M\ (N\ P)$
2. **Abstraction extends as far right as possible**: $\lambda x.M\ N$ means $\lambda x.(M\ N)$
3. **Multiple abstractions can be abbreviated**: $\lambda x.\lambda y.\lambda z.M$ can be written $\lambda xyz.M$

These conventions match how functional languages parse expressions. In Haskell, `f x y` is parsed as `(f x) y`.

Functions are everything — for real

In our Haskell lecture, we started off saying that "functions are everything." That was a slight exaggeration — Haskell has numbers, strings, data types.

On the other hand, the λ calculus makes this literally true: there is really nothing but functions...

Functions as arguments

The term $\lambda f.\lambda x.f\ x$ takes a function f and a value x, then applies f to x. In Haskell, this is just `\f x -> f x`.

Functions as return values

The term $\lambda x.\lambda y.x$ takes x and returns *a function* that ignores its argument and returns x. In Haskell: `\x -> \y -> x`.

Currying: one argument at a time

The λ calculus only allows **one argument per function**. Multi-argument functions are encoded by returning functions:

$$\lambda x.\lambda y.x + y$$

Takes x and return a function that takes y, which returns $x + y$.

Apply it step by step:

$$(\lambda x.\lambda y.x + y)\,3\,4 \;\rightarrow\; (\lambda y.3 + y)\,4 \;\rightarrow\; 3 + 4$$

This is **currying** — and works exactly as in Haskell. lication works:

```haskell
-- Haskell (curried by default)
add :: Int -> Int -> Int      -- really: Int -> (Int -> Int)
add x y = x + y

addThree :: Int -> Int        -- partial application!
addThree = add 3
```

Applying functions

Function application — the core of computation in the λ calculus — is formalized through **substitution**.

The Intuition

The expression $(\lambda x.M)\,N$ represents applying the function $\lambda x.M$ to argument N. The result should be M with every (free) occurrence of x replaced by N.

$$(\lambda x.x + x)\,5 \;\rightarrow\; 5 + 5$$

We replace x with 5 in the body $x + x$, yielding $5 + 5$. Similarly for the following example, only in two steps.

$$(\lambda x.\lambda y.x)\;5\;3 \;\rightarrow\; (\lambda y.5)\,3 \;\rightarrow\; 5$$

Simple, right? Unfortunately, naïve substitution can go wrong:

The Capture Problem. Consider the application $(\lambda x.\lambda y.x)\,y$:

- Naïvely substituting y for x in $\lambda y.x$ gives $\lambda y.y$
- But this is wrong! The outer y has been *captured* by the inner λ

To see the problem, observe that under any reasonable interpretation of λ terms, the following equality should hold:

$$\lambda x.\lambda y.x = \lambda x.\lambda z.x$$

Now, apply two equal functions to the same argument, we should obtain the same result. Unfortunately, applying the two functions to our original y gives the following:

$$(\lambda x.\lambda y.x)\,y \rightarrow \lambda y.y$$
$$(\lambda x.\lambda z.x)\,y \rightarrow \lambda z.y$$

Clearly, the results are not the same!

Solution *rename* the bound variables to avoid name clashes (α-renaming).

Formal Definitions

Free and Bound Variables

A variable occurrence in a term can be either **bound** (captured by some enclosing λ) or **free** (not captured by any λ).

For example, in $\lambda x.x\,y$:

- x is **bound** by the λ abstraction
- y is **free**—it refers to something defined elsewhere

This is analogous to lexical scoping in programming:

```
// Scala

val y = 10              // y is defined in outer scope
val f = (x: Int) => x + y   // x is bound by the lambda; y is free
```

▶ **Full details**

α-equivalence

Terms that differ only in the names of bound variables are considered identical:

$$\lambda x.x \;=\; \lambda y.y \;=\; \lambda z.z$$

Capture-free substitution

The substitution of M for all free occurrences of x in N is written

$$[M/x]N$$

To prevent any variable capture arising from a clash between the free variables in M and the bound variables in N, use α-equivalence to rename the bound variables in N before applying the substitution.

Example 1. $[M/x](\lambda x.x) = \lambda x.x$

The x inside is bound, so substitution has no effect.

Example 2. $[y/x](\lambda y.x\,y)$

Direct substitution would give $\lambda y.y\,y$—capture! We must rename first:

$$[y/x](\lambda y.x\,y) = [y/x](\lambda z.x\,z) = \lambda z.y\,z$$

Example 3. A more complex case: $[y/x]((\lambda y.x)(\lambda x.x)\,x)$

$$\begin{array}{rcl} [y/x]((\lambda y.x)(\lambda x.x)\,x) & = & [y/x](\lambda y.x)\;[y/x](\lambda x.x)\;[y/x]x \\ & = & [y/x](\lambda z.x)\;(\lambda x.x)\;y \quad \text{(rename to avoid capture)} \\ & = & (\lambda z.y)\;(\lambda x.x)\;y \end{array}$$

▶ **Full details**

Evaluation as reduction

Function application in the λ calculus is formalized as a simple rewriting, or *reduction* rule:

$$\beta\text{-reduction} \quad (\lambda x.M)\,N \quad \to \quad [N/x]M$$

We write $M \to N$ if N results from applying a β-reduction rule to M or to any subterm of M.

Evaluation is accomplished by a sequence of β-reductions. We write $M \to^* N$ to denote that M reduces to N in zero or more steps.

Redexes and evaluation strategies

Any occurrence of an application $(\lambda x.M)\,N$ inside a term is called a **redex**.

At each step, a term may contain **more than one redex**. For example:

$$((\lambda x.x)\,a)\,((\lambda y.y)\,b)$$

has two redexes (underlined below in turn):

- $(\lambda x.x)\,a$ — the left application
- $(\lambda y.y)\,b$ — the right application

Which one should we reduce first?

An **evaluation strategy** defines the **order** in which the redexes inside a term should be reduced.

The order matters more than you might expect.

Normal vs Applicative order

Not all evaluation strategies are equivalent. Two standard strategies are:

- *Normal order*: reduce the leftmost, *outermost* redex first
- *Applicative order*: reduce the leftmost, *innermost* redex first

To see how the order matters, let $\Omega = (\lambda x.x\,x)(\lambda x.x\,x)$, and consider the term $(\lambda x.y)\,\Omega$.

This term has two redexes:

- The outer redex: $(\lambda x.y)\,\Omega$
- The inner redex: Ω itself

Now:

- Normal order reduces the outer redex → converges to y

$$(\lambda x.y)\,\Omega \;\to\; y$$

- Applicative order reduces Ω first → diverges

$$(\lambda x.y)\,\Omega \;\to\; (\lambda x.y)\,\Omega \;\to\; (\lambda x.y)\,\Omega \;\to\; \cdots$$

Connection to Programming Languages

Normal and applicative order have well-established connections with the evaluation strategies adopted in programming languages.

In fact it is from normal and applicative order that such programming language strategies have originated.

λ Calculus	Evaluation	Languages
Normal order	Lazy (call-by-need)	Haskell
Applicative order	Strict (call-by-value)	Scala, Elixir, OCaml

Haskell's laziness corresponds to normal-order reduction — it always finds the normal form if one exists. Scala and Elixir use call-by-value, a proxy to applicative order (one that does not reduce under lambda's), which is more predictable in terms of memory and easier to reason about, but can diverge unnecessarily.

Normalization and Confluence

The different behavior of nornal and applicative order we just discussed — on the same term, one may converge while the other diverges — might suggest that the λ calculus is somehow inconsistent — that different evaluation strategies could yield different answers.

The Church-Rosser theorems, introduced below, dispel this concern.

Normal forms and the Church-Rosser theorems

Recall that a redex is a term of the form $(\lambda x.M)\ N$ — a function applied to an argument: a term with **no redexes** is in **normal form**.

Normal forms are the λ calculus analog of values in programming languages — they represent the final result of a computation. Then the question is:

> can a term have more than one normal form? And if a normal form exists, can we always find it?

To answer, we introcude a further concept.

Full β-reduction. Both normal-order and applicative-order are strategies — they prescribe which redex to reduce at each step. But we can also consider a non-deterministic strategy, which at each step picks *some* redex and reduce it: this is called full β-reduction, which allows any redex to be reduced at any time, in any order.

Full β-reduction enjoys the following important property

Church-Rosser Theorem I (Confluence). If $M \to^* N_1$ and $M \to^* N_2$ under full β-reduction then there exists a term P such that $N_1 \to^* P$ and $N_2 \to^* P$ (again under β-reduction)

$$\begin{tikzcd}
 & N_1 \arrow[dr, "*"] & \\
M \arrow[ur, "*"] \arrow[dr, "*"'] & & P \\
 & N_2 \arrow[ur, "*"'] &
\end{tikzcd}$$

In other words, reduction paths that diverge can always converge again. As a consequence, no term

can have two distinct normal forms — if N_1 and N_2 are both normal forms, they must be identical (up to α-equivalence).

Normal-order and applicative-order are both special cases of full β-reduction — they just make a specific choice at each step. Their relationship with full β-reduction is clarified by the second Church-Rosser theorem:

Church-Rosser Theorem II (Normalization). If a term has a normal form, normal-order reduction will find it.

Wrapping up:

- Normalization tells us that normal order is complete, whereas applicative order is not: it may diverge even when a normal form exists.
- Confluence tells us that the only way that normal order and applicative order may diverge is on non-terminating terms: for any terminating term they convervge to the same normal-form term.

The Church-Turing Thesis

The λ calculus was part of a broader effort in the 1930s to formalize the notion of "effective computation."

1933: General Recursive Functions (Gödel, Herbrand)
Functions on natural numbers built from zero, successor, and projections using composition, recursion, and minimization.

1936: λ-Definable Functions (Church)
Functions representable as λ terms

1936: Turing-Computable Functions (Turing)
Functions computable by Turing machines.

Remarkably, Church, Kleene, and Turing proved these three definitions are **equivalent**:

> A function is λ-computable ⟺ Turing-computable ⟺ general recursive

This led to the **Church-Turing Thesis**: these formal definitions capture the *intuitive* notion of "effectively calculable." Though unprovable (it relates formal to informal), it has withstood decades of attempts to find counterexamples.

The λ calculus stands as one of the three equivalent foundations of computability theory—and it's the one that most directly inspires modern functional programming.

Exercises

Exercise 1: Variable Bindings

For each expression below:

1. Fully parenthesize according to λ calculus conventions
2. Identify all **binding occurrences** (the x in λx)
3. Identify all **bound occurrences** and their binders
4. Identify all **free occurrences**

(a) $\lambda a.z\ \lambda z.a\ y$

(b) $(\lambda z.z)\ \lambda b.b\ \lambda a.a\ a$

(c) $\lambda a.\lambda b.(\lambda a.a)\ \lambda b.a$

(d) $x\ \lambda x.\lambda x.x\ (\lambda x.x)$

Exercise 2: α-Equivalence

Which of these expressions are α-equivalent (equal up to renaming)?

(a) $\lambda x.y\ \lambda a.a\ x$

(b) $\lambda x.z\ \lambda b.b\ x$

(c) $\lambda a.y\ \lambda b.b\ a$

Exercise 3: β-Reduction

Reduce each expression to normal form, showing each step:

1. $(\lambda z.z)\ (\lambda q.q\ q)\ (\lambda s.s\ a)$
2. $(\lambda z.z)\ (\lambda z.z\ z)\ (\lambda z.z\ q)$
3. $(\lambda s.\lambda q.s\ q\ q)\ (\lambda a.a)\ b$
4. $(\lambda s.\lambda q.s\ q\ q)\ (\lambda q.q)\ q$
5. $((\lambda s.s\ s)\ (\lambda q.q))\ (\lambda q.q)$

Exercise 4: Evaluation Order

Let F be a term in normal form, and Y be the following lambda term:

$$Y = \lambda f.(\lambda x.f\,(x\,x))(\lambda x.f\,(x\,x))$$

(a) Attempt to evaluate $(Y\ F)$ using **applicative order**. What happens?

(b) Evaluate $(Y\ F)$ using **normal order**. Compare to (a).

Exercise 5: Thinking Like a Functional Programmer

Consider the Haskell function:

```
take 5 [1..]  -- takes first 5 elements of infinite list
```

(a) Which evaluation strategy makes this work? Why?

(b) What would happen under the other strategy?

(c) Relate this to the $(\lambda x.y)\,\Omega$ example from the notes.

Credits

Some of the material in this lecture is adapted from *Conception, Evolution, and Application of Functional Programming Languages* by Paul Hudak, ACM Computing Surveys, 1989.